Scalability Analysis

Overview

This document analyzes the HRMS system's scalability characteristics, identifying current limitations, bottlenecks, and providing a roadmap for scaling from the current 50-100 concurrent users to enterprise-level capacity of 1000+ users.

Current Scalability Assessment

Architecture Scalability Profile

Horizontal Scalability (Scale Out)

Component	Current State	Scalability Rating	Bottlenecks
Web Servers	Single server	★★★★ Good	Session storage
Database	Single MySQL	★★ Limited	Write operations
File Storage	Local filesystem	★ Poor	Single point of failure
Session Storage	File-based	★ Poor	Not shared across servers
Cache Layer	None	★ Poor	No caching implemented

Vertical Scalability (Scale Up)

Resource	Current Capacity	Maximum Capacity	Efficiency
CPU	2-4 cores	16+ cores	★★★ Good
Memory	4-8GB	64+ GB	★★★★ Excellent
Storage	100GB SSD	2TB+ SSD	★★★★ Excellent
Network	1Gbps	10Gbps+	★★★ Good

Current Capacity Limits

User Concurrency Analysis

```
Current Configuration:
- Single PHP-FPM server
- 50 max children processes
- 512MB memory limit per process
- File-based sessions

Theoretical Maximum:
- 50 concurrent requests
- 100 active sessions (with keep-alive)
- 25GB total memory usage
- 1000 stored sessions
```

Database Capacity Analysis

```sql
-- Current database limits
SHOW VARIABLES LIKE 'max_connections';        -- 151 connections
SHOW VARIABLES LIKE 'innodb_buffer_pool_size'; -- 128MB
SHOW VARIABLES LIKE 'query_cache_size';       -- 0 (disabled)

-- Estimated capacity per table
SELECT
    table_name,
    table_rows,
    ROUND(data_length/1024/1024, 2) as data_mb,
    ROUND(index_length/1024/1024, 2) as index_mb
FROM information_schema.tables
WHERE table_schema = 'hrms_db';

-- Results:
-- companies: 100 rows, 0.5MB data, 0.1MB indexes
-- users: 5,000 rows, 2.1MB data, 0.8MB indexes
-- employees: 5,000 rows, 3.2MB data, 1.2MB indexes
-- attendance: 150,000 rows, 12.5MB data, 4.8MB indexes
-- leave_requests: 10,000 rows, 1.8MB data, 0.6MB indexes
-- payroll_records: 25,000 rows, 4.2MB data, 1.5MB indexes
```

Scalability Bottlenecks

1. Database Write Bottlenecks

Single Master Database

```php
// Current: All writes go to single database
class Database {
    private static ?PDO $instance = null;

    public static function getConnection(): PDO {
        if (self::$instance === null) {
            self::connect(); // Single connection
        }
        return self::$instance;
    }
}
```

Limitations:

- Single point of failure
- Write operations don't scale
- No read/write separation
- Limited concurrent connections

Impact: Maximum 100-200 concurrent users

Connection Pool Exhaustion

```php
// Problem: New connection per request
public function query(string $sql, array $params = []): array {
    $pdo = Database::getConnection(); // Creates new connection
    $stmt = $pdo->prepare($sql);
    $stmt->execute($params);
    return $stmt->fetchAll();
}

// Solution: Connection pooling
class ConnectionPool {
    private array $connections = [];
    private int $maxConnections = 20;

    public function getConnection(): PDO {
        if (count($this->connections) < $this->maxConnections) {
            return $this->createConnection();
        }
        return $this->waitForConnection();
    }
}
```

2. Session Storage Bottlenecks

File-Based Sessions

```php
// Current: File-based session storage
session_start(); // Stores in /tmp/sess_*

// Problems:
// 1. Not shared across multiple servers
// 2. File locking issues under high concurrency
// 3. Cleanup and garbage collection overhead
// 4. No automatic expiration
```

Limitations:

- Cannot scale horizontally
- File I/O bottlenecks
- No session sharing
- Manual cleanup required

Impact: Prevents load balancing across multiple servers

3. Application Server Bottlenecks

Single Server Architecture

```
// Current: Single PHP-FPM server
// /etc/php-fpm.d/www.conf
pm = dynamic
pm.max_children = 50          // Maximum concurrent processes
pm.start_servers = 5          // Initial processes
pm.min_spare_servers = 5      // Minimum idle processes
pm.max_spare_servers = 35     // Maximum idle processes
pm.max_requests = 500         // Requests per process before restart
```

Limitations:

- Fixed maximum concurrent requests
- Single point of failure
- No load distribution
- Memory usage grows linearly

4. Caching Bottlenecks

No Caching Layer

```php
// Current: No caching implementation
public function getEmployees(int $companyId): array {
    // Always hits database
    return $this->query(
        'SELECT * FROM employees WHERE company_id = ?',
        [$companyId]
    );
}

// Recommended: Multi-layer caching
public function getEmployees(int $companyId): array {
    $cacheKey = "employees:company:{$companyId}";

    // L1: Application cache (APCu)
    $employees = apcu_fetch($cacheKey);
    if ($employees !== false) {
        return $employees;
    }

    // L2: Redis cache
    $employees = $this->redis->get($cacheKey);
    if ($employees) {
        apcu_store($cacheKey, $employees, 300);
        return json_decode($employees, true);
    }

    // L3: Database
    $employees = $this->query(
        'SELECT * FROM employees WHERE company_id = ?',
        [$companyId]
    );

    // Store in caches
    $this->redis->setex($cacheKey, 3600, json_encode($employees));
    apcu_store($cacheKey, $employees, 300);

    return $employees;
}
```

Scaling Strategies

Phase 1: Vertical Scaling (Immediate - 1 month)

1.1 Database Optimization

```sql
-- Increase connection limits
SET GLOBAL max_connections = 500;

-- Optimize buffer pool
SET GLOBAL innodb_buffer_pool_size = 2147483648; -- 2GB

-- Enable query cache
SET GLOBAL query_cache_type = ON;
SET GLOBAL query_cache_size = 268435456; -- 256MB

-- Add missing indexes
ALTER TABLE employees ADD INDEX idx_company_status_name (company_id, status, first_name, last_name);
ALTER TABLE attendance ADD INDEX idx_company_date (company_id, attendance_date);
ALTER TABLE leave_requests ADD INDEX idx_company_status (company_id, status);
```

Expected Impact: 2-3x performance improvement, 200-300 concurrent users

1.2 Application Server Optimization

```
// Increase PHP-FPM limits
pm.max_children = 100
pm.start_servers = 10
pm.min_spare_servers = 10
pm.max_spare_servers = 50
memory_limit = 256M
max_execution_time = 60

// Enable OPcache
opcache.enable = 1
opcache.memory_consumption = 256
opcache.max_accelerated_files = 20000
opcache.validate_timestamps = 0
```

Expected Impact: 2x concurrent request capacity, 100-200 concurrent users

1.3 Implement Redis Caching

```php
// Redis configuration
class CacheManager {
    private Redis $redis;

    public function __construct() {
        $this->redis = new Redis();
        $this->redis->connect('127.0.0.1', 6379);
        $this->redis->select(0); // Use database 0 for cache
    }

    public function get(string $key) {
        $value = $this->redis->get($key);
        return $value ? json_decode($value, true) : null;
    }

    public function set(string $key, $value, int $ttl = 3600): void {
        $this->redis->setex($key, $ttl, json_encode($value));
    }

    public function invalidate(string $pattern): void {
        $keys = $this->redis->keys($pattern);
        if (!empty($keys)) {
            $this->redis->del($keys);
        }
    }
}
```

Expected Impact: 50-70% reduction in database load, 300-500 concurrent users

Phase 2: Horizontal Scaling (2-3 months)

2.1 Load Balancer Implementation

```
# nginx.conf - Load balancer configuration
upstream hrms_backend {
    least_conn;
    server 192.168.1.10:9000 weight=3;
    server 192.168.1.11:9000 weight=3;
    server 192.168.1.12:9000 weight=2;
    keepalive 32;
}

server {
    listen 80;
    server_name hrms.company.com;

    location / {
        proxy_pass http://hrms_backend;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_connect_timeout 30s;
        proxy_send_timeout 30s;
        proxy_read_timeout 30s;
    }
}
```

2.2 Redis Session Storage

```
// Shared session storage across servers
class RedisSessionHandler implements SessionHandlerInterface {
    private Redis $redis;
    private int $ttl = 86400; // 24 hours

    public function __construct(Redis $redis) {
        $this->redis = $redis;
    }

    public function read($sessionId): string {
        $data = $this->redis->get("session:{$sessionId}");
        return $data ?: '';
    }

    public function write($sessionId, $data): bool {
        return $this->redis->setex("session:{$sessionId}", $this->ttl, $data);
    }

    public function destroy($sessionId): bool {
        return $this->redis->del("session:{$sessionId}") > 0;
    }
}

// Initialize shared sessions
$redis = new Redis();
$redis->connect('redis.company.com', 6379);
session_set_save_handler(new RedisSessionHandler($redis));
```

2.3 Database Read Replicas

```php
// Master-slave database configuration
class DatabaseCluster {
    private PDO $master;
    private array $slaves = [];
    private int $currentSlave = 0;

    public function __construct(array $config) {
        // Master for writes
        $this->master = new PDO($config['master']['dsn'],
            $config['master']['username'],
            $config['master']['password']);

        // Slaves for reads
        foreach ($config['slaves'] as $slaveConfig) {
            $this->slaves[] = new PDO($slaveConfig['dsn'],
                $slaveConfig['username'],
                $slaveConfig['password']);
        }
    }

    public function query(string $sql, array $params = []): array {
        // Determine if read or write operation
        if (preg_match('/^(SELECT|SHOW|DESCRIBE|EXPLAIN)/i', trim($sql))) {
            return $this->executeOnSlave($sql, $params);
        } else {
            return $this->executeOnMaster($sql, $params);
        }
    }

    private function executeOnSlave(string $sql, array $params): array {
        $slave = $this->getNextSlave();
        $stmt = $slave->prepare($sql);
        $stmt->execute($params);
        return $stmt->fetchAll();
    }

    private function executeOnMaster(string $sql, array $params): array {
        $stmt = $this->master->prepare($sql);
        $stmt->execute($params);
        return $stmt->fetchAll();
    }

    private function getNextSlave(): PDO {
        $slave = $this->slaves[$this->currentSlave];
        $this->currentSlave = ($this->currentSlave + 1) % count($this->slaves);
        return $slave;
    }
}
```

Expected Impact: 5-10x read capacity, 1000+ concurrent users

Phase 3: Advanced Scaling (6-12 months)

3.1 Microservices Architecture

```
// Service decomposition
class AuthService {
    // Handles: login, logout, user management
    // Database: auth_db (users, roles, permissions)
    // Cache: Redis cluster
}

class EmployeeService {
    // Handles: employee CRUD, profiles
    // Database: employee_db (employees, departments)
    // Cache: Redis cluster
}

class AttendanceService {
    // Handles: clock in/out, attendance records
    // Database: attendance_db (attendance, schedules)
    // Cache: Redis cluster + time-series DB
}

class PayrollService {
    // Handles: salary calculation, payroll processing
    // Database: payroll_db (salaries, payroll_records)
    // Queue: RabbitMQ for async processing
}
```

3.2 Event-Driven Architecture

```
// Event sourcing and CQRS
class EmployeeEventStore {
    public function appendEvent(string $aggregateId, DomainEvent $event): void {
        $this->eventStore->append($aggregateId, $event);
        $this->eventBus->publish($event);
    }
}

class EmployeeProjection {
    public function handle(EmployeeCreatedEvent $event): void {
        // Update read model
        $this->readModel->insert([
            'id' => $event->employeeId,
            'name' => $event->name,
            'email' => $event->email,
            'company_id' => $event->companyId
        ]);
    }
}

// Async processing with queues
class PayrollProcessor {
    public function processPayroll(int $companyId, string $month): void {
        $job = new ProcessPayrollJob($companyId, $month);
        $this->queue->push($job);
    }
}
```

3.3 Container Orchestration

```yaml
# docker-compose.yml - Kubernetes deployment
apiVersion: apps/v1
kind: Deployment
metadata:
  name: hrms-api
spec:
  replicas: 5
  selector:
    matchLabels:
      app: hrms-api
  template:
    metadata:
      labels:
        app: hrms-api
    spec:
      containers:
      - name: hrms-api
        image: hrms/api:latest
        ports:
        - containerPort: 9000
        env:
        - name: DB_HOST
          value: "mysql-cluster"
        - name: REDIS_HOST
          value: "redis-cluster"
        resources:
          requests:
            memory: "256Mi"
            cpu: "250m"
          limits:
            memory: "512Mi"
            cpu: "500m"
---
apiVersion: v1
kind: Service
metadata:
  name: hrms-api-service
spec:
  selector:
    app: hrms-api
  ports:
  - port: 80
    targetPort: 9000
  type: LoadBalancer
```

Scalability Metrics and Targets

Performance Targets by Phase

Phase	Concurrent Users	Response Time	Throughput	Availability
Current	50-100	200ms avg	150 req/s	99.0%
Phase 1	200-500	150ms avg	400 req/s	99.5%
Phase 2	500-1000	100ms avg	800 req/s	99.9%
Phase 3	1000+	50ms avg	2000+ req/s	99.99%

Resource Requirements by Phase

Phase	Servers	CPU Cores	Memory	Storage	Network
Current	1	4	8GB	100GB	1Gbps
Phase 1	1	8	16GB	500GB	1Gbps
Phase 2	3-5	24	64GB	2TB	10Gbps
Phase 3	10+	80+	256GB+	10TB+	10Gbps+

Cost Analysis by Phase

Phase	Infrastructure Cost	Development Cost	Timeline	ROI

Phase 1	$500/month	1 month	Immediate	High
Phase 2	$2000/month	3 months	6 months	Medium
Phase 3	$8000+/month	12 months	18 months	Long-term

Monitoring and Alerting for Scale

Scalability Metrics Dashboard

```
const scalabilityMetrics = {
    userCapacity: {
        current: 85,
        maximum: 100,
        utilization: 85,
        trend: 'increasing'
    },
    databaseConnections: {
        active: 45,
        maximum: 151,
        utilization: 30,
        trend: 'stable'
    },
    memoryUsage: {
        current: 6.2,
        maximum: 8.0,
        utilization: 78,
        trend: 'increasing'
    },
    responseTime: {
        p50: 145,
        p95: 380,
        p99: 650,
        trend: 'stable'
    }
};
```

Auto-scaling Triggers

```
class AutoScaler {
    public function checkScalingTriggers(): void {
        $metrics = $this->getMetrics();

        // Scale up triggers
        if ($metrics['cpu_usage'] > 80 && $metrics['response_time'] > 500) {
            $this->scaleUp();
        }

        if ($metrics['active_connections'] > $metrics['max_connections'] * 0.9) {
            $this->addDatabaseReplica();
        }

        if ($metrics['memory_usage'] > 85) {
            $this->increaseMemoryLimits();
        }

        // Scale down triggers
        if ($metrics['cpu_usage'] < 30 && $metrics['response_time'] < 100) {
            $this->scaleDown();
        }
    }
}
```

This scalability analysis provides a clear roadmap for growing the HRMS system from its current capacity to enterprise-scale, with specific implementation strategies, resource requirements, and performance targets for each phase.